TRUEAILAB Assignment -

Production-Grade GenAI Assistant with RAG

Done by:
Raaga Prathyusha Marella
marellaraagaprathyusha@gmail.com
+91 7780678985

1. Introduction:

AI works best for complex systems, large-scale products, or problems newly solvable with modern technology. Large companies should focus on stakeholder communication, clear goals, data, and proper planning. Start-ups should prioritize fast experimentation, user feedback, and product-market fit. Common mistakes include trusting LLMs blindly, ignoring data quality, and assuming prototypes are production-ready.

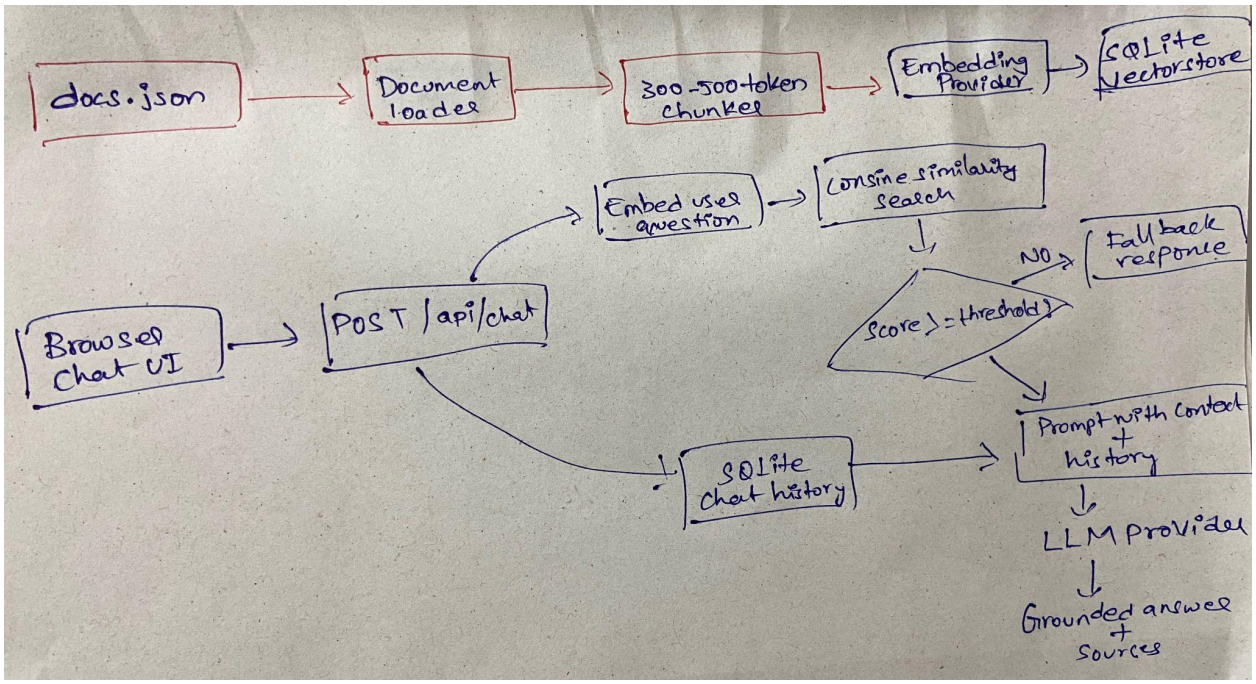
Overall, successful GenAI products require good problem definition, quality data, evaluation, and strong production processes.

2. Features:

- Loads knowledge base records from docs.json
- Chunks documents into configurable 300-500 token windows with overlap
- Generates embeddings through OpenAI, Gemini, Mistral, or local development mode
- Persists chunk text, vectors, metadata, chat sessions, and index metadata in SQLite
- Performs cosine similarity search and retrieves the top 3 chunks by default
- Applies a similarity threshold before any LLM invocation
- Injects retrieved context and short session history into a grounded prompt
- Supports OpenAI, Gemini, Claude, Mistral, or local extractive development responses
- Handles invalid payloads, provider failures, invalid keys, timeouts, and rate limits with structured errors
- Logs similarity scores and provider token usage when available

3. **Understanding:**

Builted the full RAG assistant in /Users/raaga/Desktop/RAG-assistant.
Core pieces are in place: FastAPI backend, SQLite vector store, document chunking/indexing, cosine retrieval with threshold fallback, short session memory, provider adapters for OpenAI/Gemini/Claude/Mistral, static chat frontend, sample docs.json, tests, env template, and a README with architecture/workflow docs and screenshot.



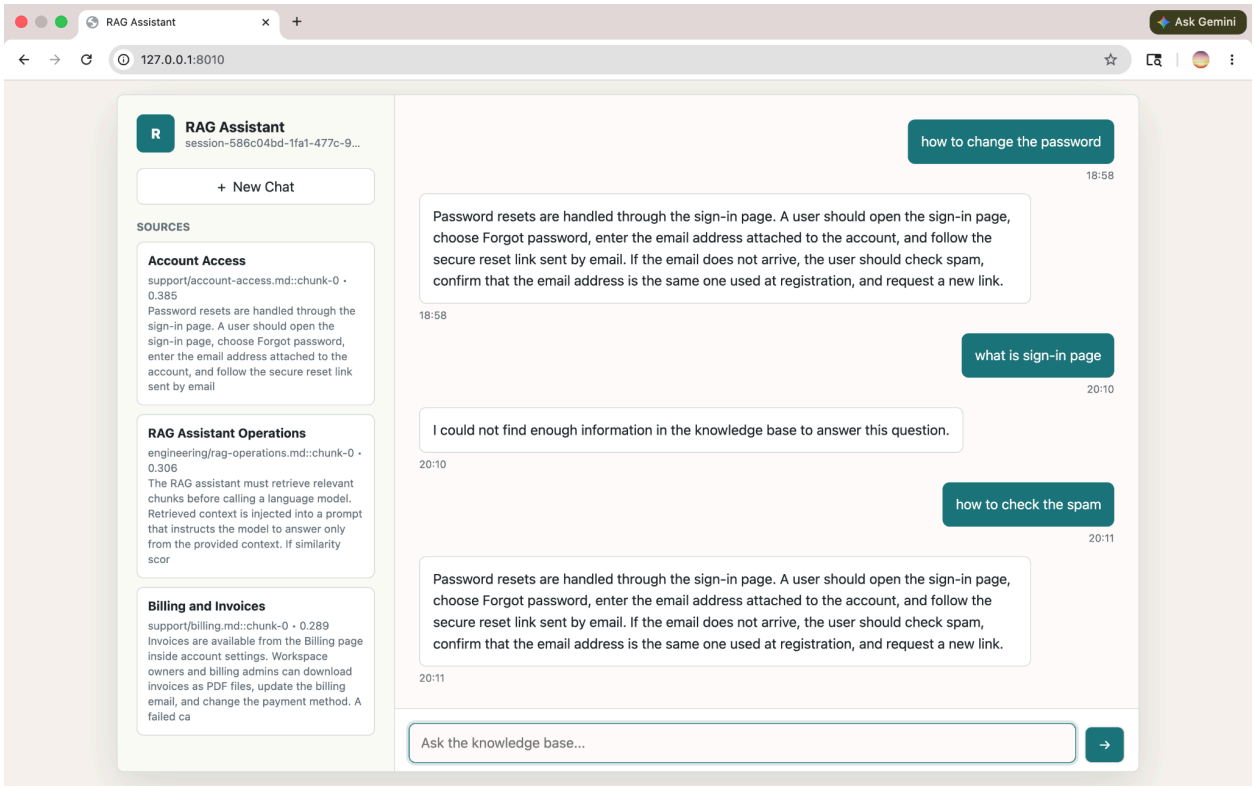
Key files:

- README.md
- app/main.py
- app/services/rag.py
- app/vectorstore/sqlite_store.py
- frontend/index.html

Verified:

- pytest → 3 passed
- GET /health → {"status":"healthy"}
- password reset query retrieves the account source
- unrelated vacation-policy query returns the required fallback
- desktop and mobile UI rendered cleanly in the browser

```
Last login: Mon May 25 15:59:06 on ttys000
raaga@RAAGAs-MacBook-Air ~ % cd ~/Desktop
raaga@RAAGAs-MacBook-Air Desktop % cd /Users/raaga/Desktop/RAG-assistant
raaga@RAAGAs-MacBook-Air RAG-assistant % pwd
/Users/raaga/Desktop/RAG-assistant
raaga@RAAGAs-MacBook-Air RAG-assistant % code .
raaga@RAAGAs-MacBook-Air RAG-assistant % python3 -m venv venv
raaga@RAAGAs-MacBook-Air RAG-assistant % source venv/bin/activate
(venv) raaga@RAAGAs-MacBook-Air RAG-assistant % tree
.
├── app
│   ├── __init__.py
│   ├── config.py
│   ├── main.py
│   ├── models
│   │   ├── __init__.py
│   │   └── schemas.py
│   ├── prompts
│   │   ├── __init__.py
│   │   └── templates.py
│   ├── routes
│   │   ├── __init__.py
│   │   ├── chat.py
│   │   └── health.py
│   ├── services
│   │   ├── __init__.py
│   │   ├── chunker.py
│   │   ├── embeddings.py
│   │   ├── indexer.py
│   │   ├── llm.py
│   │   └── rag.py
│   ├── utils
│   │   ├── __init__.py
│   │   ├── errors.py
│   │   └── logging.py
│   └── vectorstore
│       ├── __init__.py
│       └── sqlite_store.py
├── data
│   └── rag.db
├── docs
│   └── assets
│       └── chat-ui.png
├── docs.json
├── frontend
│   ├── app.js
│   ├── index.html
│   └── styles.css
├── pyproject.toml
├── README.md
├── requirements.txt
├── tests
│   ├── test_chunker.py
│   ├── test_rag.py
│   └── test_vectorstore.py
└── venv
    ├── bin
    │   ├── activate
    │   ├── activate.csh
    │   ├── activate.fish
    │   ├── Activate.ps1
    │   ├── pip
    │   ├── pip3
```



The app is running at http://127.0.0.1:8010.

Port 8000 was already occupied by another local process, so I used 8010.